

VIDYA JYOTHI INSTITUTE OF TECHNOLOGY

(Accredited by NAACA+ Approved by AICTE New Delhi & Permanently Affiliated to JNTUH)

Aziz Nagar Gate, C.B. Post, Hyderabad-500075

DEPARTMENT OF INFORMATION TECHNOLOGY

LAB PROJECT REPORT



Roll No	:	24911A1203
Name	:	A.Tejashwini
Project Title	:	Student Course Registration System
Course Name	:	Object Oriented Programming through Java Lab
Course Code	:	A224592
Academic Year	:	2025-26
Regulations	:	R22
Year and Semester	:	III B.Tech II Semester



CERTIFICATE

This is to certify that the Object Oriented Programming through Java Lab project report entitled “Student Course Registration System ” submitted by A.Tejashwini (24911A1203) to Vidya Jyothi Institute of Technology (An Autonomous Institution), Hyderabad, of B. Tech. II year, II Semester in Information Technology a Bonafide record of Lab project work carried out by them under my supervision.

Signature of Faculty

Mr. B Eswar Babu
Associate Professor

Signature of Head of the Department

Dr. A. Obulesu
Professor

Abstract

The Student Course Registration System is a Java-based software application designed to simplify and automate the process of course registration in educational institutions. In traditional manual systems, maintaining student records, course enrollment details, and fee calculations requires significant paperwork and time, which may lead to errors, duplication of data, and delays in the registration process. The main objective of this project is to provide an efficient and user-friendly system that allows students to log in securely, enroll in courses, add or drop subjects, and calculate fees automatically. The project is developed using Java programming language with Object-Oriented Programming concepts such as classes, inheritance, collections, and exception handling to ensure modularity and code reusability. Java Swing/AWT is used to create the graphical user interface, while JDBC is used for database connectivity with MySQL to store and manage student and course information efficiently. The expected outcome of this project is to reduce manual work, improve the accuracy and speed of course registration, minimize human errors, and provide better data management and security. The system also helps educational institutions maintain organized records and offers students a convenient and reliable platform for managing their academic course registrations effectively.

Table of Contents

Chapter No	Title	Page No
1	Introduction	1
2	System Analysis	2
3	System Design	4
4	Implementation	8
5	Testing	15
6	Results	18
7	Conclusion	20

Chapter 1 – Introduction

1.1 Project Overview

- The Student Course Registration System is a Java-based software application developed to automate and simplify the process of student course enrollment in educational institutions. The system allows students to log in securely, select courses, enroll in subjects, add or drop courses, and calculate fees automatically. It stores all registration details in a database using JDBC connectivity, which helps in maintaining records accurately and efficiently. The project reduces manual paperwork, minimizes human errors, and saves time during the registration process.
- In real-world applications, this system can be used in colleges, universities, schools, and training institutes to manage student registrations digitally. Administrators can maintain student and course records easily, while students can access registration services quickly and conveniently. The system improves efficiency, accuracy, and data management in educational organizations.

1.2 Objectives

- To automate the manual course registration process
- To reduce errors in student enrollment and fee calculation
- To improve the efficiency and speed of registration activities
- To provide secure storage of student and course details
- To simplify course add and drop operations
- To maintain accurate and organized student records
- To save time for both students and administrators
- To provide an easy and user-friendly registration system

1.3 Scope of the Project

- The Student Course Registration System is used by students and administrators in colleges and universities. Students can register, add or drop courses, and view fee details, while administrators can manage records efficiently. In the future, the system can be expanded by adding features like online payments, attendance tracking, notifications, and mobile app support.

Chapter 2 – System Analysis

2.1 Functional Requirements

- User login authentication
- Course enrollment
- Add courses
- Drop courses
- Fee calculation
- Store student records in database
- View registered courses
- Search student or course records
- Generate registration details and reports

2.2 Non-Functional Requirements

- Security for user login and data protection
- Fast system performance
- Reliable database storage
- User-friendly interface
- Accuracy in registration and fee calculation
- Easy maintenance and scalability

2.3 Software Requirements

Requirement	Details
Operating System	Windows/Linux
Programming Language	Java
IDE	NetBeans/Eclipse
Database	MySQL
JDK	JDK 8 or above
GUI Toolkit	Swing/AWT
Connectivity	JDBC

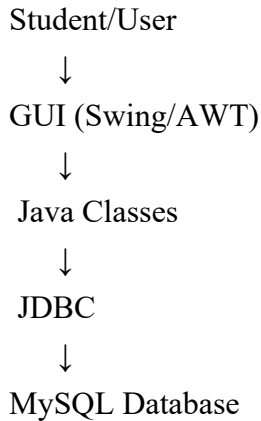
2.4 Hardware Requirements

Requirement	Details
RAM	4 GB
Processor	Intel i3 or above
Storage	500 GB
Monitor	15-inch display
Keyboard	Standard keyboard
Mouse	Optical mouse

Chapter 3 – System Design

3.1 Architecture Diagram

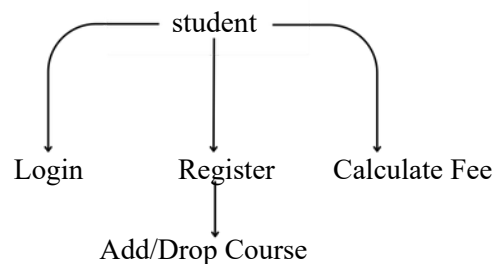
Block Diagram :



The user interacts with the GUI interface developed using Java Swing/AWT. The GUI communicates with Java classes that process course registration operations. JDBC is used to connect the application with the MySQL database for storing and retrieving student and course details.

3.2 UML Diagrams

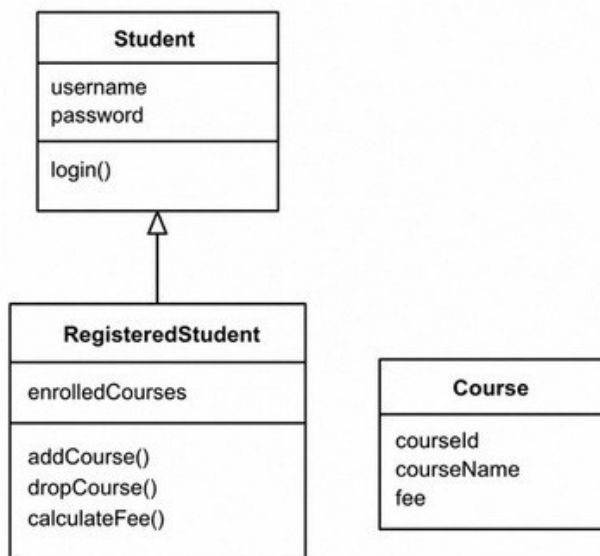
Use Case Diagram:



Main Use Cases:

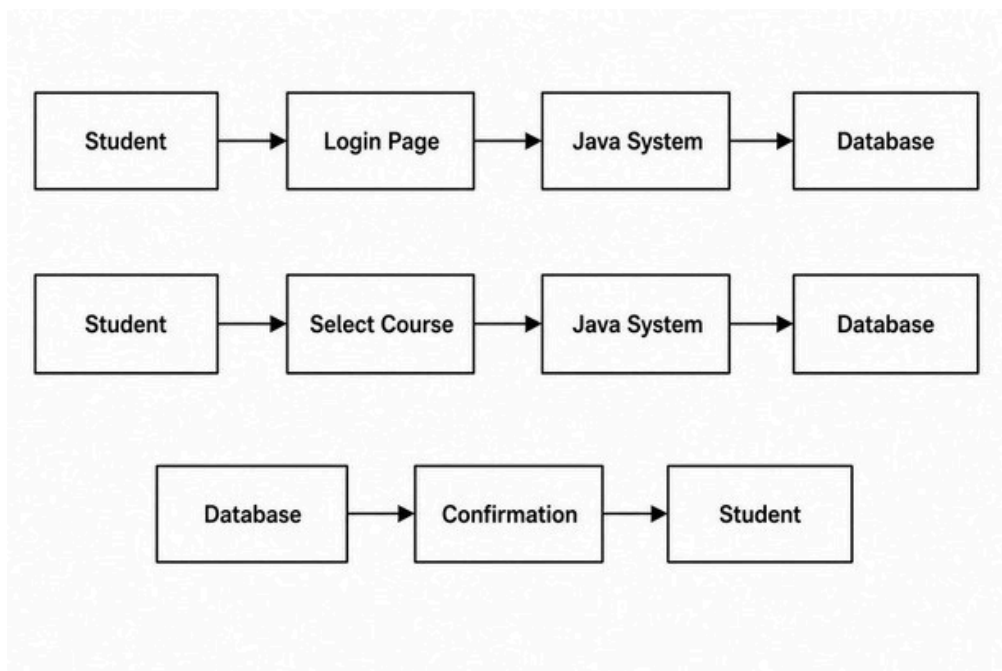
- Login into the system
- Register for courses
- Add or drop courses
- Calculate total fee
- Store registration details

Class Diagram:



- Student class handles login functionality.
- RegisteredStudent inherits properties from Student.
- Course class stores course details and fees.

Sequence Diagram:



3.3 Database Design

Tables Used :

Table Name	Purpose
students	Store student login details
courses	Store course information
registration	Store student course enrollment details
fees	Store fee calculation details

students Table:

Field Name	Data Type	Purpose
student_id	INT	Student ID
username	VARCHAR(50)	Student username
password	VARCHAR(50)	Student password

courses Table:

Field Name	Data Type	Purpose
course_id	INT	Course ID
course_name	VARCHAR(50)	Course name
fee	INT	Course fee

registration Table:

Field Name	Data Type	Purpose
reg_id	INT	Registration ID
username	VARCHAR(50)	Student username
course_name	VARCHAR(50)	Registered course

fees Table:

Field Name	Data Type	Purpose
fee_id	INT	Fee ID
username	VARCHAR(50)	Student username
total_fee	INT	Total calculated fee

Chapter 4 – Implementation

4.1 Modules Description

Module	Description
Login Module	Used for student login authentication
Course Module	Used for course enrollment and management
Add/Drop Module	Allows students to add or remove courses
Fee Module	Calculates total course fee
Database Module	Stores and retrieves records using JDBC
Report Module	Displays student registration details

4.2 Important Java Concepts Used

- Classes & Objects – Used to create Student and Course objects
- Inheritance – RegisteredStudent inherits Student class
- Polymorphism – Methods perform different operations
- Exception Handling – try-catch blocks handle errors
- Multithreading – Supports multiple users simultaneously
- Collections – ArrayList stores course details
- File Handling – Used for storing reports and records
- AWT/Swing – Used for graphical user interface
- JDBC – Used for MySQL database connectivity

4.3 Code Snippets

Database Connection Code:

```
import java.sql.*;

public class DBConnection {

    public static void main(String[] args) {

        try {

            Connection con =
                DriverManager.getConnection(
                    "jdbc:mysql://localhost:3306/studentdb",
                    "root",
                    "root");

            System.out.println(
                "Database Connected");

        } catch (Exception e) {

            System.out.println(e);

        }

    }

}
```

Database Tables Code

```
CREATE DATABASE studentdb;
```

```
USE studentdb;
```

students Table:

```
CREATE TABLE students(  
    student_id INT PRIMARY KEY AUTO_INCREMENT,  
    username VARCHAR(50),  
    password VARCHAR(50)  
);
```

courses Table:

```
CREATE TABLE courses(  
    course_id INT PRIMARY KEY,  
    course_name VARCHAR(50),  
    fee INT  
);
```

registration Table:

```
CREATE TABLE registration(  
    reg_id INT PRIMARY KEY AUTO_INCREMENT,  
    username VARCHAR(50),  
    course_name VARCHAR(50)  
);
```

Source Code:

```
import java.sql.*;
import java.util.ArrayList;

class Student {

    String username;
    String password;

    Student(String username,
            String password) {

        this.username = username;
        this.password = password;
    }

    boolean login(String u,
                  String p) {

        return username.equals(u)
            && password.equals(p);
    }
}

class Course {

    int courseId;
    String courseName;
    int fee;

    Course(int courseId,
           String courseName,
           int fee) {

        this.courseId = courseId;
        this.courseName = courseName;
        this.fee = fee;
    }
}
```

```
class RegisteredStudent
extends Student {

    ArrayList<Course>
    enrolledCourses =
    new ArrayList<>();

    RegisteredStudent(
    String username,
    String password) {

        super(username, password);
    }

    void addCourse(Course c) {

        enrolledCourses.add(c);
    }

    void dropCourse(Course c) {

        enrolledCourses.remove(c);
    }

    int calculateFee() {

        int total = 0;

        for(Course c :
        enrolledCourses) {

            total += c.fee;
        }
    }
}
```



```
    return total;
}
}
```

```
public class
StudentCourseRegistrationSystem {

    public static void main(
String[] args) {

        try {

            Connection con =
DriverManager.getConnection(
"jdbc:mysql://localhost:3306/studentdb",
"root",
"root");

            System.out.println(
"Database Connected");

            RegisteredStudent s =
new RegisteredStudent(
"admin","1234");

            if(s.login(
"admin","1234")) {

                System.out.println(
"Login Successful");
            }

            Course c1 =
new Course(
101,"Java",5000);
```

```

Course c2 =
new Course(
102,"Python",4000);

s.addCourse(c1);
s.addCourse(c2);

int total =
s.calculateFee();

System.out.println(
"Total Fee = Rs."
+ total);

String query =
"INSERT INTO registration(username, course_name) VALUES(?,?)";

PreparedStatement ps =
con.prepareStatement(query);

ps.setString(
1, s.username);

ps.setString(
2, c1.courseName);

ps.executeUpdate();

System.out.println(
"Course Registered");

con.close();

} catch(Exception e) {

System.out.println(e);
}
}
}

```

Chapter 5 – Testing

5.1 Test Cases

Test Case ID	Input	Expected Output	Result Pass
TC01	Valid Login	Login Successful	
TC02	Invalid Password	Error Message Displayed	Pass
TC03	Add Course	Course Added Successfully	Pass
TC04	Drop Course	Course Removed Successfully	Pass
TC05	Calculate Fee	Total Fee Displayed	Pass
TC06	Database Connection	Database Connected Message	Pass
TC07	Register Course	Course Registration Stored in Database	Pass

5.2 Error Handling

- SQLException – Handles database connection and SQL query errors
- NumberFormatException – Handles invalid numeric input
- NullPointerException – Handles null values in objects
- ArrayIndexOutOfBoundsException – Prevents invalid array access
- FileNotFoundException – Handles missing file errors if file handling is used
- Exception – Handles general runtime errors using try-catch blocks

Complete Process to Compile and Run Java Student Course Registration System

STEP 1: Create Folder

Create a folder:

C:\java

STEP 2: Install Required Software

Required software:

Java Development Kit

MySQL

MySQL Connector/J

STEP 3: Download MySQL Connector/J

Download from:

MySQL Connector/J Download

Choose:

Platform Independent ZIP

Extract the ZIP file.

Copy:

mysql-connector-j-9.6.0.jar

Paste into:

C:\java

STEP 4: Save Java Program

Save the program as:

StudentCourseRegistrationSystem.java

inside:

C:\java

STEP 5: Start MySQL Server

Press:

Windows + R

Type:

services.msc

Start:

MySQL80

STEP 6: Open MySQL

Open Command Prompt:mysql -u root -p

Enter password:root

STEP 7: Create Database

```
CREATE DATABASE studentdb; Use database: USE studentdb;
```

STEP 8: Create Tables

```
CREATE TABLE students(  
    student_id INT PRIMARY KEY AUTO_INCREMENT, username VARCHAR(50), password  
    VARCHAR(50));  
CREATE TABLE courses(  
    course_id INT PRIMARY KEY,  
    course_name VARCHAR(50),  
    fee INT  
);  
CREATE TABLE registration(  
    reg_id INT PRIMARY KEY AUTO_INCREMENT,  
    username VARCHAR(50),  
    course_name VARCHAR(50)  
);
```

STEP 9: Insert Values into Tables

```
INSERT INTO students(username,password)VALUES('alice','alice123'),('bob','bob123'),  
('charlie','charlie123');
```

```
INSERT INTO courses(course_id,course_name,fee)VALUES(101,'Java Programming',5000),  
(102,'Python Programming',5500),(103,'Web Development',6000);
```

```
INSERT INTO registration(username,course_name)VALUES  
('alice','Java Programming'),('bob','Python Programming'),('alice','Web Development'),  
('charlie','Java Programming');
```

STEP 10: Open CMD in Java Foldercd C:\java

STEP 11: Compile Java Program

```
javac -cp ".;mysql-connector-j-9.6.0.jar" StudentCourseRegistrationSystem.java
```

STEP 12: Run Java Program

```
java -cp ".;mysql-connector-j-9.6.0.jar" StudentCourseRegistrationSystem
```

FINAL OUTPUT

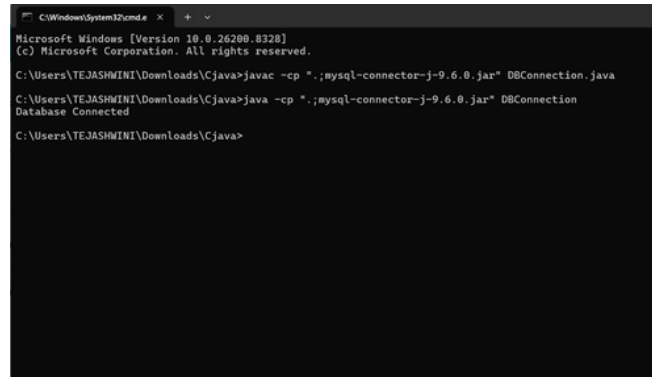
Database Connected

Login Successful

Total Fee = Rs.10500

Chapter 6 – Results and Screenshots

Database Connection Output:

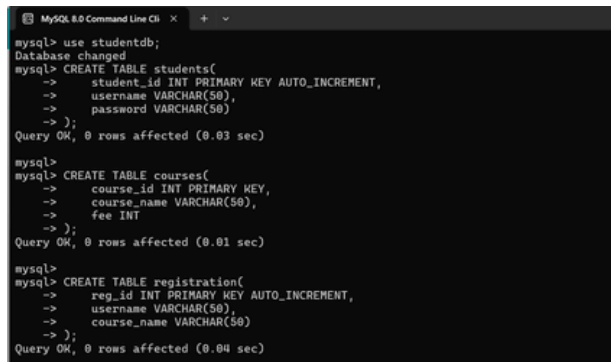


```
C:\Windows\system32\cmd.exe
Microsoft Windows [Version 10.0.26200.8328]
(c) Microsoft Corporation. All rights reserved.

C:\Users\TEJASHWINI\Downloads\CJava>javac -cp ".;mysql-connector-j-9.6.0.jar" DBConnection.java
C:\Users\TEJASHWINI\Downloads\CJava>java -cp ".;mysql-connector-j-9.6.0.jar" DBConnection
Database Connected
C:\Users\TEJASHWINI\Downloads\CJava>
```

fig1:Database connection

Database Tables Output:

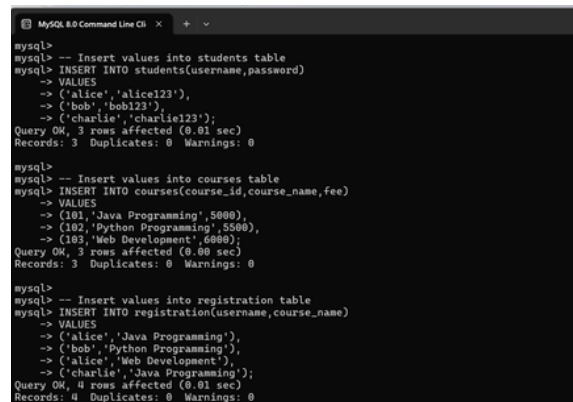


```
mysql> use studentdb;
Database changed
mysql> CREATE TABLE students(
-> student_id INT PRIMARY KEY AUTO_INCREMENT,
-> username VARCHAR(50),
-> password VARCHAR(50)
-> );
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE courses(
-> course_id INT PRIMARY KEY,
-> course_name VARCHAR(50),
-> fee INT
-> );
Query OK, 0 rows affected (0.01 sec)

mysql> CREATE TABLE registration(
-> reg_id INT PRIMARY KEY AUTO_INCREMENT,
-> username VARCHAR(50),
-> course_name VARCHAR(50)
-> );
Query OK, 0 rows affected (0.04 sec)
```

fig2.1

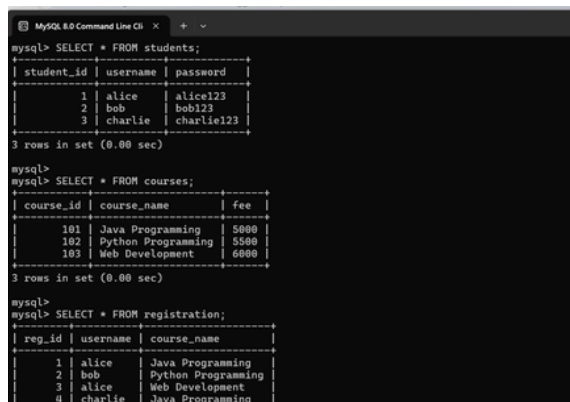


```
mysql> -- Insert values into students table
mysql> INSERT INTO students(username,password)
-> VALUES
-> ('alice','alice123'),
-> ('bob','bob123'),
-> ('charlie','charlie123');
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> -- Insert values into courses table
mysql> INSERT INTO courses(course_id,course_name,fee)
-> VALUES
-> (101,'Java Programming',5000),
-> (102,'Python Programming',5500),
-> (103,'Web Development',6000);
Query OK, 3 rows affected (0.00 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> -- Insert values into registration table
mysql> INSERT INTO registration(username,course_name)
-> VALUES
-> ('alice','Java Programming'),
-> ('bob','Python Programming'),
-> ('alice','Web Development'),
-> ('charlie','Java Programming');
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0
```

fig2.2

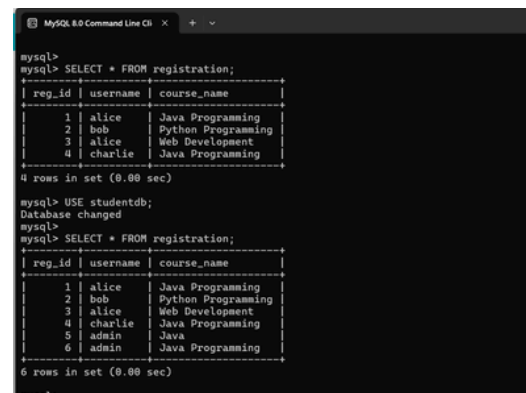


```
mysql> SELECT * FROM students;
+----+-----+-----+
| student_id | username | password |
+----+-----+-----+
| 1 | alice | alice123 |
| 2 | bob | bob123 |
| 3 | charlie | charlie123 |
+----+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM courses;
+----+-----+-----+
| course_id | course_name | fee |
+----+-----+-----+
| 101 | Java Programming | 5000 |
| 102 | Python Programming | 5500 |
| 103 | Web Development | 6000 |
+----+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM registration;
+----+-----+-----+
| reg_id | username | course_name |
+----+-----+-----+
| 1 | alice | Java Programming |
| 2 | bob | Python Programming |
| 3 | alice | Web Development |
| 4 | charlie | Java Programming |
+----+-----+-----+
4 rows in set (0.00 sec)
```

fig2.3

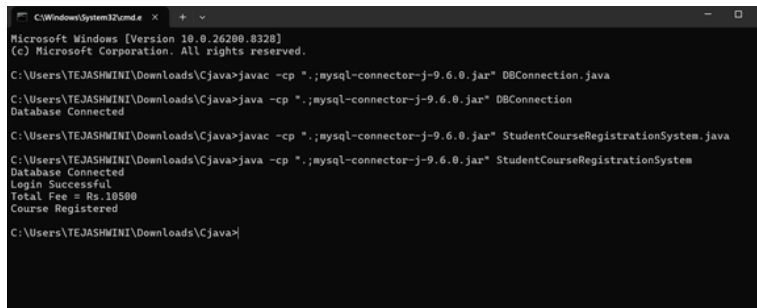


```
mysql>
mysql> USE studentdb;
Database changed
mysql> SELECT * FROM registration;
+----+-----+-----+
| reg_id | username | course_name |
+----+-----+-----+
| 1 | alice | Java Programming |
| 2 | bob | Python Programming |
| 3 | alice | Web Development |
| 4 | charlie | Java Programming |
| 5 | admin | Java |
| 6 | admin | Java Programming |
+----+-----+-----+
6 rows in set (0.00 sec)

mysql>
```

fig2.4

Source code output:



```
C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.26280.8328]
(c) Microsoft Corporation. All rights reserved.

C:\Users\TEJASHMINI\Downloads\Cjava>javac -cp ".;mysql-connector-j-9.6.0.jar" DBConnection.java

C:\Users\TEJASHMINI\Downloads\Cjava>java -cp ".;mysql-connector-j-9.6.0.jar" DBConnection
Database Connected

C:\Users\TEJASHMINI\Downloads\Cjava>javac -cp ".;mysql-connector-j-9.6.0.jar" StudentCourseRegistrationSystem.java

C:\Users\TEJASHMINI\Downloads\Cjava>java -cp ".;mysql-connector-j-9.6.0.jar" StudentCourseRegistrationSystem
Database Connected
Login Successful
Total Fee = Rs.10500
Course Registered

C:\Users\TEJASHMINI\Downloads\Cjava>
```

fig3:final output

Chapter 7 – Conclusion and Future Scope

Conclusion

The Student Course Registration System was successfully developed using Java, JDBC, and MySQL database. The project helps students to log in, enroll in courses, add or drop courses, and calculate fees easily. It reduces manual work, improves accuracy, and saves time in managing student course details. Through this project, important Java concepts such as classes, inheritance, collections, exception handling, JDBC, and Swing were learned and implemented successfully. The system works efficiently and stores data securely in the database.

Future Scope

The project can be further improved by adding advanced features such as:

- Cloud database integration
- Mobile application support
- Online payment gateway
- Email and SMS notifications
- Admin dashboard for course management
- Student profile management
- Online report generation and printing
- Multi-user login system

References

1. Java: The Complete Reference
2. Thinking in Java
3. MySQL Workbench
4. [Oracle Java Documentation](#)